Лабораторная работа № 12

Подключение к веб-сервису: чтение и разбор ответа

О чём лабораторная?

До этого момента лента объявлений в приложении работала данными, которые мы создали прямо в коде. Теперь подключим приложение к веб-сервису: получим объявления через HTTP API, разберём JSON-ответ и покажем данные в существующем `RecyclerView`.

Часть 1. Цель работы

Научиться:

- изучать документацию API и формат ответа сервера
- описывать JSON с помощью Java-моделей
- использовать Retrofit для выполнения запросов
- получать и разбирать ответ сервера
- преобразовывать данные API в модель приложения
- отображать полученный список в существующем `RecyclerView`

Часть 2. Изучаем API и проверяем ответ

Шаг 1. Откройте документацию

Перейдите по адресу:

`http://95.31.156.238:8000/docs`

Найдите операцию `GET /items` - она предназначена для получения списка объявлений.

Шаг 2. Выполните запрос

1. Раскройте операцию `GET /items`
2. Посмотрите доступные параметры
3. Нажмите `Try it out`
4. Оставьте `category_id` пустым, чтобы получить объявления без фильтра по категории
5. Нажмите `Execute`
6. Изучите статус ответа и его тело

Шаг 3. Изучите структуру ответа

Сервер возвращает не просто массив объявлений, а объект с двумя полями: `items` и `meta`.

Пример структуры ответа:

```
{
  "items": [
    {
      "id": 10,
      "title": "Гитара",
      "description": "Акустическая гитара",
      "price": 7000,
      "deal_type": "sale",
      "photo_url": null,
      "category": {
        "id": 2,
        "name": "Музыка",
        "emoji": "🎸"
      },
      "author": {
        "id": "UUID-ПОЛЬЗОВАТЕЛЯ",
        "username": "student",
        "about_me": null
      }
    }
  ],
  "meta": {
    "total": 1,
    "page": 1,
    "page_size": 20,
    "total_pages": 1
  }
}
```


Это пример структуры, обратите внимание:

- `items` - массив объявлений
- `meta` - информация о пагинации
- `category` - вложенный объект категории
- `author` - вложенный объект автора
- `photo_url` и `about_me` могут иметь значение `null`
- `deal_type` принимает одно из значений `sale`, `free` или `exchange`

Часть 3. Создаём модели для ответа сервера

На этом этапе мы ещё не отправляем запрос. Сначала описываем данные, которые приложение ожидает получить.

Для ответа API создадим отдельные классы. Это позволит не менять существующую модель `Item` и адаптер раньше времени. В проекте создайте пакет `dto` для моделей ответа API.

Шаг 1. Создайте `CategoryResponse.java`

```
public class CategoryResponse {

    private int id;
    private String name;
    private String emoji;

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getEmoji() {
        return emoji;
    }
}
```

Класс описывает вложенный объект `category` из JSON.

Шаг 2. Создайте `AuthorResponse.java`

```
public class AuthorResponse {

    private String id;
    private String username;
    private String about_me;

    public String getId() {
        return id;
    }

    public String getUsername() {
        return username;
    }

    public String getAboutMe() {
        return about_me;
    }
}
```

Класс описывает вложенный объект `author` из JSON.

Шаг 3. Создайте `ItemResponse.java`

```
public class ItemResponse {

    private int id;
```

```
    private String title;
    private String description;
    private int price;
    private String deal_type;
    private String photo_url;
    private CategoryResponse category;
    private AuthorResponse author;

    public int getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public String getDescription() {
        return description;
    }

    public int getPrice() {
        return price;
    }

    public String getDealType() {
        return deal_type;
    }

    public String getPhotoUrl() {
        return photo_url;
    }

    public CategoryResponse getCategory() {
        return category;
    }

    public AuthorResponse getAuthor() {
        return author;
    }
}
```

Здесь:

- `id`, `title`, `description`, `price` - основные данные объявления
- `deal_type` - тип объявления
- `photo_url` - адрес изображения
- `category` и `author` - вложенные объекты

В примере **имена полей совпадают с JSON.** Благодаря этому `Gson` сможет сопоставить их автоматически.

Шаг 4. Создайте `MetaResponse.java`

```
public class MetaResponse {

    private int total;
    private int page;
    private int page_size;
    private int total_pages;

    public int getTotal() {
        return total;
    }

    public int getPage() {
        return page;
    }

    public int getPageSize() {
        return page_size;
    }

    public int getTotalPages() {
        return total_pages;
    }
}
```

Класс хранит информацию о количестве записей и страниц.

Шаг 5. Создайте `PaginatedResponse.java`

Этот класс будет универсальным: параметр типа `T` позволяет использовать его для страниц с объектами разных типов.

```
public class PaginatedResponse<T> {

    private List<T> items;
    private MetaResponse meta;

    public List<T> getItems() {
        return items;
    }

    public MetaResponse getMeta() {
        return meta;
    }
}
```

Класс содержит список объектов `items` типа `T` и метаданные пагинации `meta`.

Теперь мы описали всю структуру ответа.

Часть 4. Подключаем Retrofit

Retrofit позволяет описывать HTTP-запросы в виде Java-интерфейсов, а Gson преобразует JSON-ответ в созданные нами модели.

Шаг 1. Добавьте зависимости

Откройте файл `build.gradle.kts (Module :app)`.

Внутри блока `dependencies` добавьте:

```
implementation("com.squareup.retrofit2:retrofit:2.12.0")
implementation("com.squareup.retrofit2:converter-gson:2.12.0")
implementation("com.squareup.okhttp3:logging-interceptor:3.14.9")
```

Нажмите `Sync Now` и дождитесь завершения синхронизации Gradle.

Шаг 2. Разрешите сетевые запросы

Откройте `AndroidManifest.xml` и добавьте разрешение перед тегом `<application>`:

```
<uses-permission android:name="android.permission.INTERNET" />
```

Наш сервер использует адрес с `http://`, потому нужно добавить в существующий тег `<application>`:

```
android:usesCleartextTraffic="true"
```

Шаг 3. Создайте `RetrofitClient.java`

В пакете `network` создайте класс-клиент:

```
public class RetrofitClient {

    private static final String BASE_URL = "http://95.31.156.238:8000/";

    private static Retrofit retrofit;

    public static Retrofit getRetrofit() {
        if (retrofit == null) {
            HttpLoggingInterceptor logging =
                    new HttpLoggingInterceptor();

            logging.setLevel(
                    HttpLoggingInterceptor.Level.BODY
            );

            OkHttpClient client = new OkHttpClient.Builder()
                    .addInterceptor(logging)
```

```
                    .build();

            retrofit = new Retrofit.Builder()
                    .baseUrl(BASE_URL)
                    .client(client)
                    .addConverterFactory(
                            GsonConverterFactory.create()
                    )
                    .build();
        }

        return retrofit;
    }

    public static ApiService getApiService() {
        return getRetrofit().create(ApiService.class);
    }
}
```

Часть 5. Описываем запрос в API-интерфейсе

Теперь, когда мы знаем структуру ответа и создали соответствующие модели, можно описать запрос.

Шаг 1. Создайте `ApiService.java`

```
public interface ApiService {

    @GET("items")
    Call<PaginatedResponse<ItemResponse>> getItems(
        @Query("page") int page,
        @Query("page_size") int pageSize,
        @Query("category_id") Integer categoryId
    );
}
```

Разберём код:

- `@GET("items")` - отправить GET-запрос по пути `/items`
- `@Query("page")` - передать номер страницы
- `@Query("page_size")` - задать размер страницы
- `@Query("category_id")` - передать необязательный фильтр
- `Call<PaginatedResponse<ItemResponse>>` - получить ответ, который преобразуется в объект `PaginatedResponse<ItemResponse>`

Часть 6. Выполняем запрос и обрабатываем ответ

Шаг 1. Выполните запрос асинхронно

Сетевой запрос нельзя выполнять в главном потоке интерфейса, потому используем `enqueue()`:

```
ApiService apiService = RetrofitClient.getApiService();

apiService.getItems(1, 20, null).enqueue(new
Callback<PaginatedResponse<ItemResponse>>() {

    @Override
    public void onResponse(
        Call<PaginatedResponse<ItemResponse>> call,
        Response<PaginatedResponse<ItemResponse>> response
    ) {
        if (response.isSuccessful() && response.body() != null) {

            PaginatedResponse<ItemResponse> result = response.body();
            List<ItemResponse> items = result.getItems();
            MetaResponse meta = result.getMeta();

            // здесь можно обработать полученный  список

        } else {
            // попали сюда так как сервер вернул HTTP-ошибку или отсутствует тело
ответа
        }
    }

    @Override
    public void onFailure(
        Call<PaginatedResponse<ItemResponse>> call,
        Throwable t
    ) {
        // ошибка соединения или выполнения запроса
    }
});
```

Шаг 2. Преобразуем ответ API в модель приложения

В проекте уже есть класс `Item`, который используется локальными объявлениями и адаптером `ItemAdapter`.

`ItemResponse` описывает ответ **сервера**, а `Item` - данные, которые использует **приложение**. Эти классы **не обязаны совпадать**: например, сервер возвращает вложенные объекты `category` и `author`, а модель приложения может хранить название владельца и числовой идентификатор категории.

Создайте отдельный метод преобразования внутри класса `ItemResponse`. Например, логика преобразования может выглядеть так:

```
private static Item mapToItem(ItemResponse response) {
    int categoryId = response.getCategory() != null
            ? response.getCategory().getId()
            : -1;
```

```
    String owner = response.getAuthor() != null
            ? response.getAuthor().getUsername()
            : "Неизвестный автор";

    String imageUri = response.getPhotoUrl();

    DealType dealType = DealType.SALE;

    if ("free".equals(response.getDealType())) {
        dealType = DealType.FREE;
    } else if ("exchange".equals(response.getDealType())) {
        dealType = DealType.EXCHANGE;
    }

    return new Item(
        response.getId(),
        dealType,
        categoryId,
        response.getTitle(),
        response.getPrice(),
        response.getDescription(),
        owner,
        imageUri
    );
}
```

Часть 7. Подключаем полученный список к RecyclerView

Откройте `MainActivity`.

1. Оставьте существующую настройку `RecyclerView`, `GridLayoutManager` и `ItemAdapter`
2. Создайте метод `loadItemsFromServer()`
3. Внутри него выполните запрос `getItems(1, 20, null)`
4. После успешного ответа получите список `ItemResponse`
5. Преобразуйте каждый объект с помощью `mapToItem()`
6. Передайте новый список адаптеру
7. Обработайте пустой список и ошибки

Пример преобразования списка:

```
List<Item> mappedItems = new ArrayList<>();

for (ItemResponse itemResponse : items) {
    mappedItems.add(mapToItem(itemResponse));
}
```

Далее обновите адаптер в соответствии с его реализацией. Если адаптер хранит неизменяемый список, добавьте метод для обновления данных либо создайте новый адаптер с новым списком.

Требования

- Проект успешно синхронизируется и собирается
- Retrofit и Gson подключены
- Модели соответствуют структуре ответа API
- Запрос выполняется к `GET /items`
- Полученный JSON преобразуется в `ItemsResponse`
- Список `items` преобразуется в модель `Item`
- Объявления отображаются в существующем `RecyclerView`

Дополнительно

1. Добавьте загрузку следующей страницы объявлений
2. Добавьте индикатор загрузки
3. Добавьте кнопку повторной загрузки при сетевой ошибке
4. Покажите в интерфейсе общее количество объявлений из `meta.total`